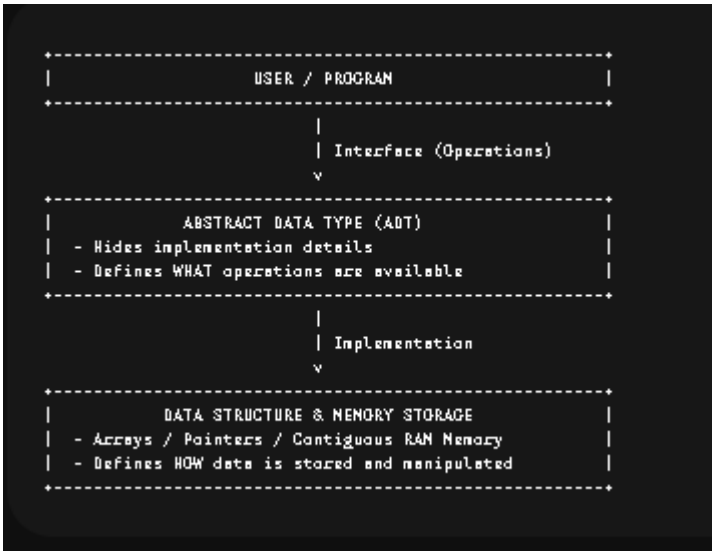
DATA STRUCTURES & ALGORITHM ANALYSIS

1. Data Abstraction and Algorithms

A. Data Abstraction

- **Deep Definition:** Data Abstraction is the technique of hiding internal implementation details of data organization and exposing only an essential interface to the user.
- **Levels of Abstraction:**
    1. **Abstract Level (What it does):** Defines the mathematical model of the data and the set of operations permitted on it (e.g., a List supports insert, delete, search).
    2. **Concrete / Implementation Level (How it does it):** Specifies how data is physically arranged in memory (e.g., using contiguous arrays vs. dynamically allocated linked nodes) and the specific algorithms executing the operations.



B. Algorithms

- **Definition:** An algorithm is a finite, step-by-step, unambiguous sequence of computational instructions that takes an input value (or set of values) and produces a correct output value in a finite amount of time.
- **Essential Properties of an Algorithm:**
    - **Input:** Zero or more externally supplied quantities.
    - **Output:** At least one processed result produced.
    - **Definiteness:** Every instruction must be clear, unambiguous, and precise.
    - **Finiteness:** Must terminate after a finite number of steps for all execution cases.
    - **Effectiveness:** Every operation must be sufficiently basic to be executed in practice using finite resources.

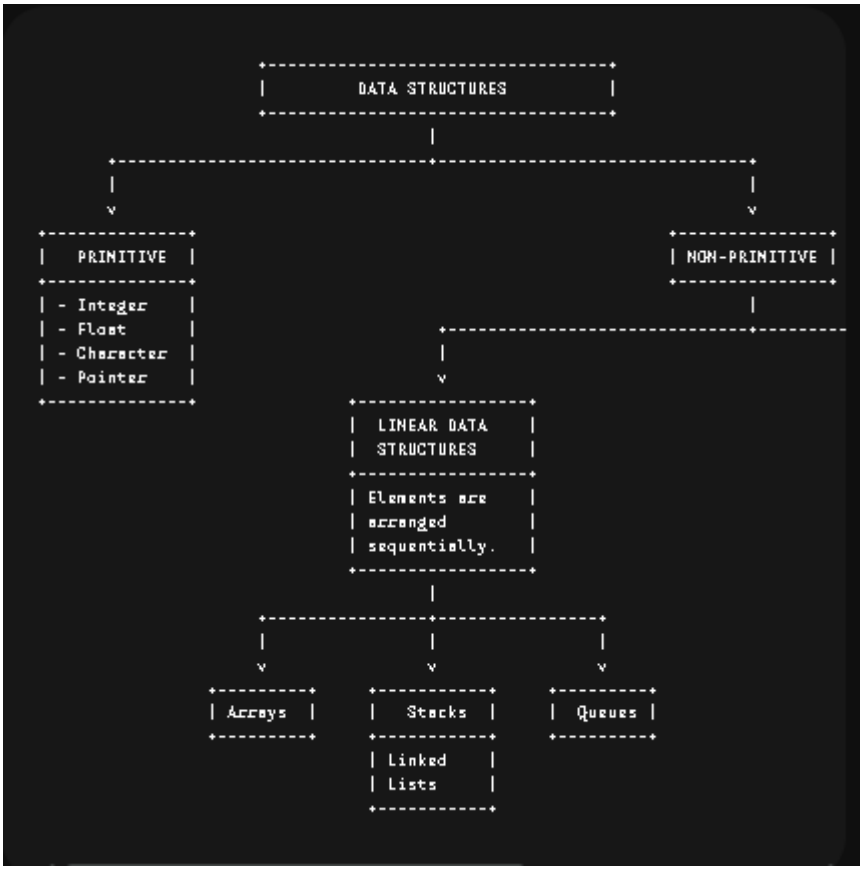
2. Data Types, Data Objects, Data Structures, and ADTs

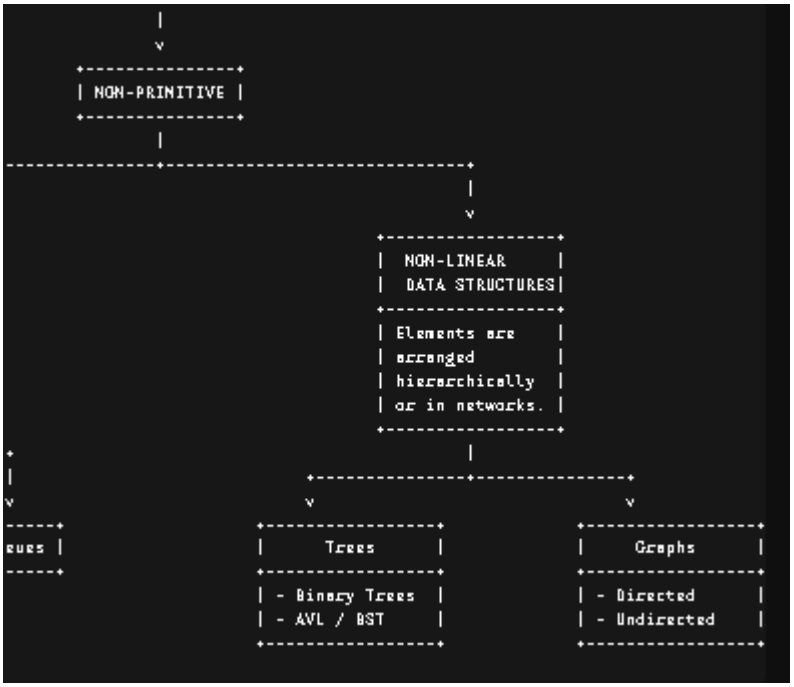
To avoid confusion in computer science theory, these four fundamental concepts are strictly differentiated:

Concept	Formal Definition	Examples

Data Type	A domain of values together with a set of operations permitted on those values, enforced by a programming language compiler or interpreter.	int, float, char, double
Data Object	A concrete region of storage containing a value of a specific data type; an instance of a data type.	int x = 10; (here x is the object)
Data Structure	A specialized format for organizing, processing, storing, and retrieving data in physical computer memory to enable efficient algorithmic operations.	Array, Linked List, Binary Tree, Hash Table
Abstract Data Type (ADT)	A mathematical specification of a set of data objects and the operations that can be performed on them, defined independently of implementation details.	Stack ADT, Queue ADT, Priority Queue ADT

3. Classification of Data Structures





4. Algorithm Analysis & Asymptotic Notations

- **Deep Definition:** Algorithm analysis evaluates the computational efficiency of an algorithm as a function of input size n. It allows comparison of algorithms independent of programming language, compiler optimizations, or execution hardware.

Asymptotic Notations (Growth Rates):

1. **Big-O Notation (O) – Upper Bound (Worst-Case):**
   - *Definition:* Expresses the maximum growth rate of an algorithm's execution time or memory.
   - *Mathematical Formulation:* $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that:
   - $0 \leq f(n) \leq c \cdot g(n) for all n \geq n_0$
2. **Big-Omega Notation (Ω) – Lower Bound (Best-Case):**
   - *Definition:* Expresses the minimum growth rate of an algorithm.
   - *Mathematical Formulation:* $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that:
   - $0 \leq c \cdot g(n) \leq f(n) for all n \geq n_0$
3. **Big-Theta Notation (Θ) – Tight Bound (Average / Exact Case):**
   - *Definition:* Bound an algorithm both above and below within constant factors.
   - *Mathematical Formulation:* $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that:
   - $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) for all n \geq n_0$

Common Growth Rates Ranked from Fastest to Slowest:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

5. Time Requirements of Algorithms (Time Complexity)

- **Deep Definition:** Time complexity measures how the execution time of an algorithm grows as the input size n increases.

- **Counting Operation Frequency:** Instead of measuring runtime in seconds (which varies by hardware), time complexity is computed by counting fundamental operations (comparisons, assignments, additions).

Step-by-Step Code Analysis Examples:

Example 1: Single Loop — $O(n)$ Time

```
C
int sum = 0;
for (int i = 0; i < n; i++) {
   sum += i; // Executed n times
}
```

- **Analysis:** The loop executes n times. Each iteration takes $O(1)$ constant time operations.
- **Total Time Complexity:** $T(n) = O(n)$ (Linear Time).

Example 2: Nested Loops — $O(n^2)$ Time

```
C
int count = 0;
for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
      count++; // Executed n * n times
   }
}
```

- **Analysis:** Outer loop runs n times; for each outer iteration, the inner loop runs n times.
- **Total Execution Steps:** $n \times n = n^2$.
- **Total Time Complexity:** $T(n) = O(n^2)$ (Quadratic Time).

Example 3: Logarithmic Loop — $O(\log n)$ Time

```
C
int i = 1;
while (i < n) {
   i = i * 2; // Value doubles each iteration
}
```

- **Analysis:** In iteration k, the value of i is 2^k. The loop terminates when $2^k \ge n \implies k = \log_2 n$.
- **Total Time Complexity:** $T(n) = O(\log n)$ (Logarithmic Time).

6. Space Requirements of Algorithms (Space Complexity)

- **Deep Definition:** Space complexity measures the total memory space required by an algorithm to run to completion as a function of input size n.
- **Components of Total Memory Space:**
    - $TotalSpaceS(P) = FixedSpaceRequirements(C) + VariableSpaceRequirements(S_M(I))$
    - **Fixed Space (C):** Independent of input parameters. Includes code space, simple variable storage, and static constants.
    - **Variable Space ($S_M(I)$):** Dynamic memory space dependent on input size n. Includes dynamically allocated memory on the heap (e.g., malloc) and recursion stack frames on the call stack.

Analyzing Space Complexity:

Example 1: Iterative Sum — $O(1)$ Auxiliary Space

```
C
int sumArray(int arr[], int n) {
    int total = 0; // 4 bytes
    for (int i = 0; i < n; i++) { // 4 bytes for loop counter
        total += arr[i];
    }
    return total;
}
```

- **Analysis:** The input array occupies $O(n)$ space. However, the **auxiliary space** (extra memory used beyond the input) consists only of scalar variables (total, i), requiring constant memory.
- **Auxiliary Space Complexity:** $O(1)$ (Constant Space).

Example 2: Recursive Factorial — $O(n)$ Auxiliary Space

```
C
int factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1); // Generates call stack frame
}
```

- **Call Stack Trace for factorial(4):**
  factorial(4) → factorial(3) → factorial(2) → factorial(1)
- **Analysis:** The maximum recursion depth is n. The system stack must maintain n distinct call stack frames simultaneously in memory to preserve local parameters and return addresses.
- **Auxiliary Space Complexity:** $O(n)$ (Linear Stack Space).

ARRAYS, ADDRESS CALCULATIONS, & SPARSE MATRIX REPRESENTATION

1. Array Representation & Memory Layout

A. Single Dimensional Arrays (1D)

- **Deep Definition:** A 1D array is a linear sequence of homogeneous data elements stored in contiguous memory locations.
- **Element Indexing:** If an array A starts at base memory address B, with lower bound LB and element size W bytes, the physical address of element $A[i]$ is calculated using the formula:

$$Address(A[i]) = B + W \times (i - LB)$$

B. Multidimensional Arrays (2D Mapping Formulas)

Computers have a linear 1D memory space. To store a 2D matrix $A[m][n]$ (with m rows and n columns), two primary mapping schemes are used:

```
        2D MATRIX LAYOUT
         Column 0  Column 1  Column 2
Row 0   [   a00,      a01,      a02   ]
Row 1   [   a10,      a11,      a12   ]
```

1. ROW-MAJOR ORDER (Store row by row sequentially):
Memory: [a00 | a01 | a02 | a10 | a11 | a12]

2. COLUMN-MAJOR ORDER (Store column by column sequentially):
Memory: [a00 | a10 | a01 | a11 | a02 | a12]

1. Row-Major Order (RMO)

- **Concept:** Matrix elements are stored row by row sequentially.
- **Address Formula for $A[i][j]$:**
    - $Address(A[i][j]) = B + W \times [(i - L_r) \times N_c + (j - L_c)]$
- where $B = BaseAddress$, $W = ElementSize$, $L_r, L_c = LowerBoundsofrowsandcolumns$, $N_c = Totalnumberofcolumns$.

2. Column-Major Order (CMO)

- **Concept:** Matrix elements are stored column by column sequentially.
- **Address Formula for $A[i][j]$:**
    - $Address(A[i][j]) = B + W \times [(j - L_c) \times N_r + (i - L_r)]$
- where $N_r = Totalnumberofrows$.

C. Step-by-Step Solved Address Calculation Example

Problem: Given a 2D array $A[1\dots10][1\dots15]$ with Base Address $B = 2000$ and element size $W = 4$ bytes. Calculate the address of $A[6][8]$ using both Row-Major and Column-Major order.

- *Given values:* $L_r = 1, L_c = 1, N_r = 10, N_c = 15$, Target $i = 6, j = 8$.
- **Solution (Row-Major Order):**
    - $Address(A[6][8]) = 2000 + 4 \times [(6 - 1) \times 15 + (8 - 1)]$
- $= 2000 + 4 \times [5 \times 15 + 7] = 2000 + 4 \times [75 + 7] = 2000 + 4 \times 82 = 2000 + 328 = 2328$
- **Solution (Column-Major Order):**
    - $Address(A[6][8]) = 2000 + 4 \times [(8 - 1) \times 10 + (6 - 1)]$
- $= 2000 + 4 \times [7 \times 10 + 5] = 2000 + 4 \times [70 + 5] = 2000 + 4 \times 75 = 2000 + 300 = 2300$

2. Specialized Sparse & Structured Matrices (Vector Representation)

A **Sparse Matrix** is a matrix in which the majority of elements are zero. Storing all elements in a standard 2D array wastes memory. Instead, structured sparse matrices can be compressed into 1D vectors using indexing functions.

A. Lower Triangular Matrix

- **Definition:** A square matrix $A_{n \times n}$ where all elements *above* the main diagonal are zero ($a_{ij} = 0$ for $i < j$).
- **Non-zero Element Count:** $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$.

Lower Triangular Matrix (4x4):

[a11 0   0   0]
[a21 a22 0   0]
[a31 a32 a33 0]
[a41 a42 a43 a44]

- **1D Vector Mapping Formula (Row-Major Storage):**
  To map element $A[i][j]$ (using 1-based indexing where $i \geq j$) into a 1D vector V:

  - $IndexinVectorV = \frac{i(i-1)}{2} + (j - 1)$

B. Upper Triangular Matrix

- **Definition:** A square matrix $A_{n\times n}$ where all elements *below* the main diagonal are zero ($a_{ij} = 0$ for $i > j$).
- **Non-zero Element Count:** $\frac{n(n+1)}{2}$.

Upper Triangular Matrix (4x4):
[a11 a12 a13 a14]
[0   a22 a23 a24]
[0   0   a33 a34]
[0   0   0   a44]

- **1D Vector Mapping Formula (Row-Major Storage):**
  To map element $A[i][j]$ (where $i \leq j$) into a 1D vector V:

  - $IndexinVectorV = \left[(i - 1)n - \frac{(i-1)(i-2)}{2}\right] + (j - i)$

C. Tridiagonal Matrix

- **Definition:** A square matrix $A_{n\times n}$ where all non-zero elements reside exclusively on the main diagonal, the sub-diagonal (directly below main diagonal), and the super-diagonal (directly above main diagonal). Thus, $a_{ij} = 0$ if $\vert{}i - j\vert{} > 1$.
- **Non-zero Element Count:** $n + (n - 1) + (n - 1) = 3n - 2.$

Tridiagonal Matrix (4x4):
[d1   u1   0   0] <- Row 1: 2 non-zero elements
[l1   d2   u2   0] <- Row 2: 3 non-zero elements
[0   l2   d3   u3] <- Row 3: 3 non-zero elements
[0   0   l3   d4] <- Row 4: 2 non-zero elements

- **1D Vector Mapping Scheme:**
  A Tridiagonal matrix $A_{n\times n}$ is stored efficiently in a 1D vector V of size $3n - 2$ by mapping diagonals sequentially:
  - **Lower Sub-diagonal ($i - j = 1$):** Stored first from index 0 to $n - 2$.
  - **Main Diagonal ($i - j = 0$):** Stored next from index $n - 1$ to $2n - 2$.
  - **Upper Super-diagonal ($i - j = -1$):** Stored last from index $2n - 1$ to $3n - 3$.
- **Mapping Equations for $A[i][j]$ (1-based indexing):**
  - Case 1: $i - j = 1$ (Lower diagonal) $\implies \text{Index} = i - 2$
  - Case 2: $i - j = 0$ (Main diagonal) $\implies \text{Index} = (n - 1) + (i - 1)$
  - Case 3: $i - j = -1$ (Upper diagonal) $\implies \text{Index} = (2n - 1) + (i - 1)$

3. General Sparse Matrix: 3-Tuple Representation & C Program

For arbitrary sparse matrices, the **3-Tuple Representation** stores non-zero elements as a list of triplets: (Row, Column, Value).

C

```
#include <stdio.h>

#define MAX 100

// Structure to store non-zero sparse matrix elements
typedef struct {
    int row;
    int col;
    int val;
} Element;

void convertTo3Tuple(int matrix[4][5], int rows, int cols, Element sparse[]) {
    int k = 1; // Index for sparse representation (sparse[0] holds metadata)

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] != 0) {
                sparse[k].row = i;
                sparse[k].col = j;
                sparse[k].val = matrix[i][j];
                k++;
            }
        }
    }
    // Metadata row storing total dimensions and non-zero count
    sparse[0].row = rows;
    sparse[0].col = cols;
    sparse[0].val = k - 1;
}

void printSparse(Element sparse[]) {
    int total = sparse[0].val;
    printf("\n--- 3-TUPLE SPARSE MATRIX REPRESENTATION ---\n");
    printf("Row\tCol\tValue\n");
    printf("------------------------\n");
    for (int i = 0; i <= total; i++) {
        printf("%d\t%d\t%d\n", sparse[i].row, sparse[i].col, sparse[i].val);
    }
}

int main() {
    int matrix[4][5] = {
        {0, 0, 3, 0, 4},
        {0, 0, 0, 0, 0},
        {0, 2, 0, 0, 0},
        {5, 0, 0, 0, 0}
    };
```

```
    Element sparse[MAX];
    convertTo3Tuple(matrix, 4, 5, sparse);
    printSparse(sparse);

    return 0;
}
```

STACKS, QUEUES, AND LINKED LISTS

PART 1: STACKS (ADT & EXPRESSION CONVERSIONS)

1. Introduction & Primitive Operations on Stack

- **Deep Definition:** A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The element inserted most recently is the first one removed. Access is restricted to only one end of the stack, called the **Top**.
- **Primitive Operations:**
    1. push(item): Inserts an element at the top of the stack. (Time Complexity: $O(1)$)
    2. pop(): Removes and returns the top element from the stack. (Time Complexity: $O(1)$)
    3. peek() / top(): Returns the top element without removing it. (Time Complexity: $O(1)$)
    4. isEmpty(): Checks if the stack contains any elements.
    5. isFull(): Checks if the stack has reached its maximum capacity (applicable to static array implementations).

Stack Implementation using C Structure & Array:

C

```
#include <stdio.h>
#define MAX 100

typedef struct {
    int items[MAX];
    int top;
} Stack;

void initStack(Stack *s) {
    s->top = -1;
}

int isEmpty(Stack *s) {
    return s->top == -1;
}

int isFull(Stack *s) {
    return s->top == MAX - 1;
}

void push(Stack *s, int val) {
    if (isFull(s)) {
        printf("Stack Overflow!\n");
        return;
    }
    s->items[++(s->top)] = val;
}
```

```
int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return s->items[(s->top)--];
}
```

2. Applications of Stacks

1. **Expression Evaluation & Conversion:** Converting between Infix, Prefix, and Postfix notation.
2. **Function Call Management (Call Stack):** Managing local variables, return addresses, and parameters during recursive and nested function calls.
3. **Undo Mechanisms:** Storing user actions in applications (e.g., text editors, drawing software) to reverse them.
4. **Parentheses Matching:** Checking for balanced grouping symbols (), {}, [] in source code compilers.
5. **Backtracking Algorithms:** Pathfinding, maze solving, and graph traversal (DFS).

3. Infix, Postfix, and Prefix Expressions

- **Infix Notation:** Operators are written *between* operands (e.g., $A + B$). Requires parentheses to define precedence order.
- **Postfix Notation (Reverse Polish Notation - RPN):** Operators are written *after* their operands (e.g., $AB +$). Requires no parentheses.
- **Prefix Notation (Polish Notation):** Operators are written *before* their operands (e.g., $+ AB$).

4. Evaluation of Postfix Expression

- **Algorithm:**
    - Scan the postfix expression from left to right.
    - If an operand is encountered, push it onto the stack.
    - If an operator is encountered, pop the top two operands from the stack ($op2$ first, then $op1$), evaluate op1 operator op2, and push the result back onto the stack.
    - When the expression ends, the final value remaining on the stack is the result.
- **Solved Numerical:** Evaluate postfix expression: 5 3 + 8 2 - *
    - **Trace Table:**

Token	Action	Stack Contents (Bottom to Top)
5	Push operand	[5]
3	Push operand	[5, 3]
+	Pop 3 and 5, compute $5 + 3 = 8$, push	[8]

8	Push operand	[8, 8]
2	Push operand	[8, 8, 2]
-	Pop 2 and 8, compute $8 - 2 = 6$, push	[8, 6]
*	Pop 6 and 8, compute $8 \times 6 = 48$, push	[48]

- **Final Result: 48**.

5. Conversion from Infix to Postfix Expression

- **Operator Precedence Rules:**
  - ^ (Highest, Right-to-Left)
  - *, / (Medium, Left-to-Right)
  - +, - (Lowest, Left-to-Right)
- **Algorithm:**
  - Scan infix expression from left to right.
  - If operand, output it directly to the postfix string.
  - If (, push onto stack.
  - If), pop from stack and output until corresponding (is popped.
  - If an operator is encountered, pop operators from the stack that have ≥ precedence and output them, then push the current operator.
  - At end of expression, pop all remaining stack operators.
- **Solved Numerical:** Convert Infix $A + B * C - D/E$ to Postfix.
  - **Trace Table:**

Token	**Stack**	**Output Postfix Expression**
A	Empty	A
+	[+]	A
B	[+]	A B
*	[+, *] (* has higher precedence than +)	A B

C	[+, *]	A B C
-	[+] (* popped), then [-, -] -> wait, pop * first, push -	A B C * +
D	[-]	A B C * + D
/	[-, /] (/ has higher precedence than -)	A B C * + D
E	[-, /]	A B C * + D E
End	Empty (pop all)	A B C * + D E / -

- **Final Postfix:** A B C * + D E / -

PART 2: QUEUES & ADVANCED QUEUES

1. Introduction & Primitive Operations on Queues

- **Deep Definition:** A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle. Elements are added at one end called the **Rear** (Enqueue) and removed from the other end called the **Front** (Dequeue).
- **Primitive Operations:**
    1. enqueue(item): Inserts an element at the rear of the queue.
    2. dequeue(): Removes and returns the front element from the queue.
    3. peek(): Returns the front element without removing it.
    4. isEmpty() / isFull(): Status checkers.

Linear Queue Limitation:
In a linear array-based queue, once the rear reaches the maximum array index, no further elements can be enqueued, even if spaces at the front became available due to prior dequeues (resulting in wasted memory). This is solved by the **Circular Queue**.

2. Deques (Double-Ended Queues) and Priority Queues

- **Deque (Double-Ended Queue):** A generalized queue where insertion and deletion operations are permitted at **both** ends (Front and Rear). Can be restricted to Input-restricted or Output-restricted variants.
- **Priority Queue:** A specialized queue where each element is associated with a priority. Elements are served based on their priority rather than their arrival sequence (Higher priority elements are dequeued before lower priority ones).

PART 3: LINKED LISTS

1. Introduction & Memory Architecture

- **Deep Definition:** A **Linked List** is a linear collection of data elements called **Nodes**, where each node points to the next node via a memory address pointer, rather than being stored in contiguous memory locations (unlike arrays).
- **Anatomy of a Node:**
- C

```
struct Node {
   int data;          // Data payload
   struct Node* next;  // Pointer to the next node in memory
};
```

-
-

2. Variations of Linked Lists

1. **Singly Linked List:** Each node points to the next node. Traversal is strictly unidirectional (forward).
2. **Circular Linked List:** The last node points back to the first node (head) instead of NULL, forming a continuous loop.
3. **Doubly Linked List:** Each node contains *two* pointers: next (pointing to the succeeding node) and prev (pointing to the preceding node), enabling bidirectional traversal.

3. Comprehensive Singly Linked List Implementation & Operations

A. Node Structure & Initialization

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

// Function to create a new node dynamically
struct Node* createNode(int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->next = NULL;
   return newNode;
}
```

B. Traversal Operation

C

```
void traverseList(struct Node* head) {
   struct Node* ptr = head;
   printf("Linked List: ");
   while (ptr != NULL) {
      printf("%d -> ", ptr->data);
      ptr = ptr->next;
   }
   printf("NULL\n");
}
```

C. Insertion Operations

1. **Insertion at Beginning:**
2. C

```
void insertAtBegin(struct Node** head, int val) {
    struct Node* newNode = createNode(val);
    newNode->next = *head;
    *head = newNode;
}
```

3.
4.
5. **Insertion at End:**
6. C

```
void insertAtEnd(struct Node** head, int val) {
    struct Node* newNode = createNode(val);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* ptr = *head;
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }
    ptr->next = newNode;
}
```

7.
8.

D. Deletion Operation

C

```
void deleteNode(struct Node** head, int key) {
    struct Node *temp = *head, *prev = NULL;

    // If head node itself holds the key to be deleted
    if (temp != NULL && temp->data == key) {
        *head = temp->next; // Changed head
        free(temp);         // Free old head
        return;
    }

    // Search for the key to be deleted
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }

    // If key was not present in linked list
    if (temp == NULL) return;

    // Unlink the node from linked list
    prev->next = temp->next;
    free(temp);
}
```

E. Searching Operation

C

```
int searchNode(struct Node* head, int key) {
    struct Node* current = head;
    int position = 1;
    while (current != NULL) {
        if (current->data == key)
            return position;
        current = current->next;
        position++;
    }
    return -1; // Not found
}
```

4. Applications of Linked Lists

1. **Dynamic Memory Allocation:** Used when the maximum data size is unknown beforehand (unlike fixed-size arrays).
2. **Implementation of Stacks and Queues:** Linked lists provide dynamic resizing without overflow limitations.
3. **Graph Adjacency Lists:** Representing vertices and weighted edges in graph algorithms.
4. **Polynomial Arithmetic:** Representing terms with coefficients and exponents dynamically.
5. **Browser History:** Doubly linked lists tracking forward and backward web page navigation.

TREES AND GRAPHS

PART 1: TREES AND ADVANCED TREE STRUCTURES

1. Introduction and Terminology

- **Deep Definition:** A **Tree** is a non-linear hierarchical data structure consisting of a collection of nodes linked together, representing a parent-child relationship. It has a single root node and zero or more subtrees.
- **Core Terminologies:**
    - **Root:** The topmost node of a tree from which all other nodes descend.
    - **Parent & Child:** Every node (except the root) has a directed connection from a unique parent node. Nodes originating from a parent are its children.
    - **Leaf (External Node):** A node that has no children.
    - **Internal Node:** A node with at least one child.
    - **Height of a Node:** The length of the longest path from that node down to a leaf.
    - **Depth (Level) of a Node:** The length of the path from the root node to that specific node.

2. Traversal of Binary Trees

Visiting every node in a tree exactly once in a specific systematic order is called traversal. For **Binary Trees** (where each node has at most two children), there are three primary Depth-First traversal strategies:

1. **Inorder Traversal (**$Left \rightarrow Root \rightarrow Right$**):**
    - *Process:* Traverse the left subtree, visit the root, then traverse the right subtree. (In a BST, this yields sorted ascending order).
2. **Preorder Traversal (**$Root \rightarrow Left \rightarrow Right$**):**
    - *Process:* Visit the root first, then traverse the left subtree, and finally the right subtree. (Used for creating a copy of a tree or prefix expressions).
3. **Postorder Traversal (**$Left \rightarrow Right \rightarrow Root$**):**

- *Process:* Traverse the left subtree, traverse the right subtree, and visit the root last. (Used for deleting a tree or bottom-up calculations).

3. Binary Search Tree (BST) & Recursive Operations

- **Definition:** A **Binary Search Tree** is a binary tree where every node satisfies the BST property:
  - The value of all keys in the left subtree of a node is *less than* the node's key.
  - The value of all keys in the right subtree of a node is *greater than* the node's key.

Recursive Operations in C (BST):

```
C
#include <stdio.h>
#include <stdlib.h>

struct BSTNode {
    int data;
    struct BSTNode *left, *right;
};

// Create a new node
struct BSTNode* createNode(int val) {
    struct BSTNode* newNode = (struct BSTNode*)malloc(sizeof(struct BSTNode));
    newNode->data = val;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// 1. Recursive Insertion
struct BSTNode* insert(struct BSTNode* root, int val) {
    if (root == NULL) return createNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else if (val > root->data)
        root->right = insert(root->right, val);
    return root;
}

// 2. Inorder Traversal
void inorder(struct BSTNode* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

4. AVL Trees (Self-Balancing Binary Search Trees)

- **Deep Definition:** An **AVL Tree** (Adelson-Velsky and Landis) is a self-balancing binary search tree where the difference between the heights of the left and right subtrees (called the **Balance Factor**) for *every* node is no greater than 1.

  - $BalanceFactor = Height(LeftSubtree) - Height(RightSubtree) \in \{-1, 0, 1\}$

- **Tree Rotations:** When an insertion or deletion causes a node's balance factor to become -2 or $+2$, the tree restructures itself using rotations:
    1. **LL Rotation (Left-Left):** Single right rotation.
    2. **RR Rotation (Right-Right):** Single left rotation.
    3. **LR Rotation (Left-Right):** Left rotation on left child, followed by right rotation on root.
    4. **RL Rotation (Right-Left):** Right rotation on right child, followed by left rotation on root.

PART 2: GRAPHS

1. Definition, Terminologies, and Properties

- **Deep Definition:** A **Graph** $G = (V, E)$ is a non-linear data structure consisting of a set of vertices (V) and a set of edges (E) connecting pairs of vertices.
- **Core Terminologies:**
    - **Directed vs. Undirected Graph:** In undirected graphs, edges have no orientation (two-way); in directed graphs (digraphs), edges have a specific direction.
    - **Weighted Graph:** Edges carry numerical values representing costs, distances, or capacities.
    - **Degree of a Vertex:** The number of edges connected to a vertex. (In directed graphs, split into *In-degree* and *Out-degree*).
    - **Path & Cycle:** A sequence of edges connecting a sequence of vertices. A **cycle** is a path that starts and ends at the exact same vertex.

2. Graph Representations

Computers store graphs primarily using two standard formats:

A. Adjacency Matrix

A 2D array A of size $\vert{}V\vert{} \times \vert{}V\vert{}$ where entry $A[i][j] = 1$ (or edge weight) if an edge exists from vertex i to vertex j, and 0 otherwise.

- *Space Complexity:* $O(V^2)$ (Efficient for dense graphs).

B. Adjacency List

An array of linked lists. Each vertex maintains a linked list of its directly connected neighboring vertices.

- *Space Complexity:* $O(V + E)$ (Efficient for sparse graphs).

3. Graph Traversal Algorithms

A. Breadth-First Search (BFS)

- **Concept:** Explores the graph level-by-level, visiting all immediate neighbors of a node before moving to the next level. Implemented using a **Queue**.
- **Applications:** Finding shortest paths in unweighted graphs, peer-to-peer networks.

B. Depth-First Search (DFS)

- **Concept:** Explores as deep as possible along each branch before backtracking. Implemented using a **Stack** (or recursion).
- **Applications:** Cycle detection, topological sorting, solving mazes.

4. Minimum Spanning Trees (MST)

- **Definition:** Given a connected, undirected, weighted graph, a **Minimum Spanning Tree** is a subgraph that connects all vertices together with the minimum possible total edge weight, without forming any cycles (it is a tree).
- **Classic Algorithms:**

1. **Kruskal's Algorithm:** Greedily adds the lowest weight edges one by one, using a Disjoint Set Union (DSU) data structure to prevent cycles.
2. **Prim's Algorithm:** Starts from an arbitrary vertex and greedily grows the MST by adding the minimum weight edge connecting the current tree to an unvisited vertex.

SORTING, SEARCHING, AND HASHING

PART 1: INTERNAL VS. EXTERNAL SORTING

- **Internal Sorting:** Occurs when all data items to be sorted are small enough to fit entirely into the computer's primary main memory (RAM) at once. Operations are extremely fast since memory access is direct. (Examples: Quick Sort, Merge Sort, Heap Sort).
- **External Sorting:** Occurs when the dataset is too massive to fit into RAM (e.g., gigabytes or terabytes of records). The algorithm must store the bulk of data on slow secondary storage devices (HDDs or SSDs) and bring blocks into memory sequentially for processing. **Merge Sort** is the foundational algorithm used in external sorting (External Merge Sort).

PART 2: ADVANCED SORTING ALGORITHMS

1. Heap Sort

- **Deep Definition:** A comparison-based sorting technique based on a **Binary Heap** data structure (specifically a Max-Heap where parent nodes are greater than or equal to their children).
- **Algorithm Steps:**
    1. Build a Max-Heap from the unordered input array.
    2. Swap the root element (maximum value) with the last element of the heap.
    3. Reduce the heap size by 1 and call maxHeapify on the new root to restore the heap property.
    4. Repeat steps 2 and 3 until the entire array is sorted.
- **Time Complexity:** $O(n \log n)$ in all cases (Best, Average, Worst).
- **Space Complexity:** $O(1)$ auxiliary space (in-place sorting).

2. Merge Sort

- **Deep Definition:** A classic **Divide and Conquer** algorithm. It divides the array into two halves, recursively sorts them, and then merges the two sorted halves back together.
- **Algorithm (Divide and Conquer):**
    1. **Divide:** Find the midpoint mid of the array and split it into left and right sub-arrays.
    2. **Conquer:** Recursively sort both sub-arrays using Merge Sort.
    3. **Combine:** Merge the two sorted sub-arrays into a single sorted array.
- **Time Complexity:** $O(n \log n)$ across all cases.
- **Space Complexity:** $O(n)$ auxiliary space required for temporary merging arrays.

C Implementation of Merge Function:

C

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

3. Quick Sort

- **Deep Definition:** Another efficient **Divide and Conquer** sorting algorithm. It selects a 'pivot' element and partitions the array such that all elements smaller than the pivot go to its left, and all larger elements go to its right.
- **Algorithm & Partitioning (Lomuto Scheme):**
    - Choose a pivot (e.g., the last element).
    - Rearrange the array so elements smaller than the pivot are placed before it, and larger elements after it.
    - Recursively apply the above steps to the sub-arrays on the left and right of the pivot.
- **Time Complexity:**
    - Average & Best Case: $O(n \log n)$
    - Worst Case (poor pivot selection): $O(n^2)$
- **Space Complexity:** $O(\log n)$ due to the recursive call stack.

4. General Radix Sort

- **Deep Definition:** A **non-comparison-based** sorting algorithm that sorts data with integer keys by grouping keys by individual digits sharing the same significant position and value.
- **Mechanism:** Radix Sort processes digits from the **Least Significant Digit (LSD)** to the **Most Significant Digit (MSD)** using a stable sorting algorithm (like Counting Sort) as a subroutine.
- **Time Complexity:** $O(d \cdot (n + b))$ where n is the number of items, d is the number of digits, and b is the base (e.g., base 10).

PART 3: SEARCHING ALGORITHMS

1. Sequential (Linear) Search

- **Concept:** Iterates through each element of the array sequentially from the beginning until the target element is found or the list ends.
- **Data Requirement:** Works on both sorted and unsorted lists.
- **Time Complexity:** Best Case $O(1)$, Worst/Average Case $O(n)$.

2. Binary Search

- **Concept:** An efficient search algorithm that works on **sorted** arrays. It repeatedly divides the search interval in half.
- **Algorithm:**
    1. Find the middle index $mid = low + (high - low)/2$.
    2. If the target equals arr[mid], return the index.
    3. If the target is less than arr[mid], search the left half (high = mid - 1).
    4. If the target is greater than arr[mid], search the right half (low = mid + 1).
- **Time Complexity:** $O(\log n)$ (Logarithmic time).

C Implementation of Binary Search:

C

```
int binarySearch(int arr[], int size, int target) {
    int low = 0, high = size - 1;
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // Element not found
}
```

PART 4: HASHING, HASH FUNCTIONS, & COLLISION RESOLUTION

- **Deep Definition:** Hashing is a technique used to map data of arbitrary size to fixed-size values (keys) using a mathematical algorithm called a **Hash Function**, allowing near-instantaneous $O(1)$ average-time data retrieval.

1. Hash Functions

A good hash function should be easy to compute, distribute keys uniformly across the hash table, and minimize collisions. Common methods include:

- **Division Method (Modulo):** $\text{Hash}(K) = K \pmod M$ (where M is the size of the hash table, preferably a prime number).
- **Multiplication Method:** $\text{Hash}(K) = \lfloor M \times (K \cdot A \pmod 1) \rfloor$ (where $0 < A < 1$).
- **Mid-Square Method:** Square the key value K, then extract a designated number of middle digits as the hash address.

2. Collision Resolution Techniques

A **collision** occurs when two distinct keys hash to the exact same table index. There are two primary categories of resolution strategies:

A. Closed Addressing (Separate Chaining)

- **Concept:** Each index of the hash table points to a linked list of entries that hashed to that same index.
- **Advantages:** Simple to implement, table never truly fills up, graceful degradation under heavy load.

B. Open Addressing

All elements are stored directly within the hash table itself. When a collision occurs, alternative slots are probed using specific sequences:

1. **Linear Probing:**
    - Checks the next sequential slot if a collision occurs:
      $$\text{Index}(K, i) = (\text{Hash}(K) + i) \pmod M \quad \text{for } i = 0, 1, 2, \dots$$
    - *Drawback:* Prone to **clustering** (consecutive filled slots forming blocks, slowing down search times).
2. **Quadratic Probing:**
    - Uses a quadratic polynomial to resolve clustering:
      $$\text{Index}(K, i) = (\text{Hash}(K) + c_1 \cdot i + c_2 \cdot i^2) \pmod M$$
3. **Double Hashing:**
    - Uses a secondary hash function to calculate the probe step size, eliminating primary and secondary clustering:
      $$\text{Index}(K, i) = (\text{Hash}_1(K) + i \cdot \text{Hash}_2(K)) \pmod M$$